

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.DOI

Collaborative Privacy Preserving Industrial Anomaly Detection using U-Shaped Split Learning with Blockchain-Based Service Logging

SURYA PRAKASH GARLAPATI¹, SURABHI R.¹, TANISHA SRIVASTAVA¹, VINAYAK KUMAR SINGH², N. YAMINI³, and ANALA M. R.²

¹Department of Computer Science and Engineering, RV College of Engineering, Bengaluru, Karnataka 560059, India

²Department of Information Science and Engineering, RV College of Engineering, Bengaluru, Karnataka 560059, India

³Department of Artificial Intelligence and Machine Learning, RV College of Engineering, Bengaluru, Karnataka 560059, India

Corresponding author: Anala M. R. (e-mail: analamr@rvce.edu.in).

ABSTRACT Industries increasingly incline on using machine learning for anomaly detection and predictive maintenance. However, developing highly accurate and robust models almost always require data from multiple industries, while privacy concerns, intellectual property protection, and regulations prevent the direct sharing of proprietary industrial data. This paper proposes a collaborative privacy preserving anomaly detection framework based on U-shaped Split Learning and blockchain-enabled service logging. The proposed architecture allows multiple industrial clients to collaboratively train a shared anomaly detection model without exposing raw sensor data or labels. A 1D Convolutional Neural Network is partitioned between client nodes and a central server, where only intermediate activations are exchanged during training. This design reduces privacy risks while maintaining effective model learning. The framework is evaluated using the Bosch Research CNC Machining dataset containing tri-axial sensor measurements. Experimental results demonstrate that the proposed approach achieves an anomaly detection accuracy of approximately (~87.4%) while significantly reducing information leakage and label inference vulnerability. In addition, a Solana-based blockchain layer provides secure, immutable, and transparent logging of anomaly-triggered maintenance tickets. The proposed system enables organizations to collaboratively build robust anomaly detection models while preserving data confidentiality, making it suitable for Industrial Internet of Things (IIoT) environments.

INDEX TERMS Anomaly Detection, Blockchain, Federated Split Learning, Industrial IoT, Predictive Maintenance, Privacy-Preserving Machine Learning, Solana, Split Learning, Vibration Analysis.

I. INTRODUCTION

The growth and expansion of Industry 4.0 technologies has led to an increased the deployment of sensors and Industrial Internet of Things (IIoT) devices for continuous monitoring of manufacturing equipment health. Modern production environments generate large volumes of vibration, temperature, acoustic, and process signals that can be utilized for fault diagnosis of machines and predictive maintainance. This in turn enables timely response before catastrophic failures occur. It is estimated that the world's largest manufacturing companies lose about \$1.5 trillion every year due to production downtime or failure of production equipment. As per a

blog by Seimens, in the automotive industry, each hour of lost production costs more than \$2 million [1]. As a result of this large scale of economic loss and the implementation of these new technologies, intelligent condition monitoring is emerging as one of the leading areas of research and engineering within modern industrial systems.

Deep learning frameworks especially Convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTM) networks, and their hybrids have been proven to be successful in extracting discriminative features directly from raw vibration signals. This can help bypass the need for any manual feature engineering. [2], [3]. 1D CNNs are very

beneficial in handling time-series vibration data because of low parameter requirements, translation invariance, and the capability to pull multi-scale temporal features associated with both normal and abnormal operating conditions. [4]. However, deploying these models across multiple organizations remains difficult in practice, since maximizing data utility and protecting data privacy pull in opposite directions..

Centralized conventional training requires raw operational data to be sent and stored on one central server. This raises concerns regarding data privacy, intellectual property protection and regulatory compliance (e.g. CCPA, GDPR). It also adds to communication overhead. Often, the sensor measurements used to generate time-series vibration data contain sensitive information related to machine setup (e.g. equipment configuration), production schedule, and equipment utilization patterns. As a result, many companies are unwilling or legally unable to share their raw data across their organization boundaries to form silos. As the diversity of data decreases quality of the trained model will also reduce.

Federated Learning addresses this by having clients exchange only gradient updates instead of raw data, making it one of the most widely adopted approaches to distributed privacy-preserving training.. [5]. However this is computationally prohibitive for resource-constrained IIoT edge devices that must simultaneously perform real-time sensing. This puts excessive strain on clients with limited computational capabilities. Furthermore, gradient sharing is vulnerable to gradient inversion attacks. An attacker can reconstruct the original training data from intercepted gradient updates exchanged between the client and the federated learning server. [6]

To reduce this burden, Vepakomma et al. [7] introduced Split Learning, which splits the network at a chosen layer so computation is shared between client and server rather than fully duplicated. Clients execute only the lightweight front portion of the network and transmit intermediate activation tensors to the server. The server processes the remaining layers of the network, computes the loss, and returns gradients to the client. This substantially reduces client-side computation. However, conventional SL requires clients to share ground-truth labels with the server for loss computation. This exposes sensitive information to a potentially untrusted server. Furthermore the returned gradients may leak label dependent information through gradient inversion attacks [8], [9].

Motivated by these limitations, this paper proposes a Privacy-Preserving U-Shape Split Learning (USL) framework for industrial fault diagnosis. By adopting a U-shaped network topology, both the front-end feature extractor and the back-end classifier remain on the client and the server performs an intermediate feature refinement role. Loss computation is thus performed entirely on the client using local labels, eliminating the need to transmit supervisory information to the server and removing the server's ability to compute label-dependent gradients.

To further strengthen the system's integrity and auditability properties, we integrate a Solana blockchain component

that records maintenance events as immutable, append-only transactions whenever an anomaly is detected. This integration solves a complementary problem, even if the learning phase is privacy-preserving, the resulting maintenance records (fault identifiers, timestamps, service outcomes) are vulnerable to tampering if stored in conventional mutable databases. Blockchain's immutability and transparency provide a cryptographically verifiable audit trail for accountability in the absence of a fully trusted central authority.

The main inferences from this work are as follows:

- **U-Shaped Split Learning for CNC Fault Diagnosis**
- **Complete Label and Gradient Privacy**
- **Comparison of Normal and U-Shaped Split Learning**
- **End-to-End Automated Fault Lifecycle Verification using Blockchain**

The rest of this paper is organized as below. Section II reviews the related literature across four dimensions: deep learning for industrial fault diagnosis, split and federated learning, privacy attacks and defenses, and blockchain-assisted IIoT systems. Section III describes the proposed methodology, including dataset preparation, model architecture, training protocols, privacy evaluation metrics, and blockchain integration. Experimental results are presented and analyzed in Section IV. Conclusion and direction for future research is given in Section V.

II. RELATED WORK

A. A. DEEP LEARNING FOR INDUSTRIAL FAULT DIAGNOSIS

Deep learning has emerged as the state-of-the-art approach for vibration-based fault diagnosis. The transition from manually engineered signal-processing methods to automated learning frameworks based on CNNs and autoencoders was highlighted by Lei et al. [2]. Building on this progress, Niu et al. [3] proposed multitask residual CNNs with cross-sensor information fusion, while Zhang et al. [4] applied 1D-CNNs directly to raw vibration signals, achieving high diagnostic accuracy with minimal preprocessing. Furthermore, Zhao et al. [10] explored transformer-based architectures for related predictive maintenance tasks, such as remaining useful life estimation. Despite these advancements, most existing approaches depend on centrally aggregated training data, an assumption that is often impractical in multi-organization Industrial IoT (IIoT) environments due to privacy, intellectual property, and regulatory concerns. To address this limitation, our work enables collaborative model training without requiring the centralization of raw vibration data.

B. SPLIT LEARNING AND FEDERATED LEARNING

Federated Learning, proposed by McMahan et al. [5], keeps raw data on-device and instead transmits only gradient updates.. This technique was extended to intelligent transportation by Otoum et al. [11] and to predictive maintenance by Bharti and McGibney [12]. However, the requirement

in FL that each client trains a complete model is often not feasible for resource-constrained sensors. To address this issue, Vepakomma et al. [7] proposed SL, which transfers only a lightweight front network to the client side. Extensive experimental evaluations by Hu et al. [13], as well as wireless and robot-oriented implementations by Wu et al. [14] and Ni et al. [15], demonstrate the computational efficiency of SL. However, these studies indicate that conventional SL requires the transfer of labels to the server, which can expose supervisory information. To combine the advantages of both paradigms, Thapa et al. [16], Hafi et al. [17], and Sindhura et al. [18] introduced hybrid SplitFed architectures; however, these models inherit the same label-exposure weakness. Our work is most closely related to that of Tang et al. [19], who demonstrated that a U-shaped topology reduces the amount of information observable by the server without significantly increasing communication overhead.

C. PRIVACY ATTACKS AND DEFENSES

FL and SL both continue to face major privacy vulnerabilities. Gradient reconstruction attacks were successfully demonstrated by Zhao et al. [6], Zhu et al. [8], and Yin et al. [9]. Moreover, Yang et al. [20] and Liu et al. [21] revealed vulnerabilities to model-inversion and label-inference attacks based on intermediate activations and label transmissions. To counter these security threats, homomorphic encryption was proposed by Lin et al. [22], differential privacy noise was applied by Hafi et al. [17], while Alhindi et al. [23] and Khan et al. [24] explored architectural modifications. Despite their strong theoretical guarantees, encryption-based techniques are computationally expensive and therefore unsuitable for resource-constrained edge devices. Alternatively, the U-shaped architecture proposed by Vepakomma et al. [7] prevents label and gradient leakage by relocating loss computation to the client side. In this work, we studied and quantitatively evaluated this approach for high-frequency industrial signals, a domain that remains underexplored in the split learning privacy literature.

D. BLOCKCHAIN FOR IIOT SECURITY

Blockchains have found to have broad application in delivering decentralized trust and tamper-resistant logging for IIoT. For example, Yazdinejad et al. [25] used blockchain for federated threat hunting, while Weng et al. [26] and Miao et al. [27] leveraged blockchain to ensure provable lineage of model updates. Chen et al. [28] proposed a traceable industrial data-sharing framework, whereas Ahmad et al. [29] and Jin et al. [30] developed tamper-resistant maintenance audit trails. However, many blockchain platforms suffer from latency or cost overheads that are prohibitively expensive for real-time maintenance logging tasks. Yakovenko [31] introduced Solana's Proof-of-History protocol, which provides sub-second finality and near-zero transaction costs, making it suitable for this purpose. To the best of our knowledge, it has not previously been combined with privacy-preserving split

learning in an end-to-end industrial fault diagnosis pipeline, which constitutes the key contribution of our work.

Based on these observations, this study is guided by the following research questions:

RQ1: Can a U-shaped split learning topology match or exceed the classification accuracy of conventional split learning for industrial vibration-based fault diagnosis?

RQ2: To what extent does relocating loss computation to the client reduce quantifiable privacy leakage—measured via Information Exposure Ratio, Gradient Inversion Risk Score, and reconstruction NMSE—relative to conventional split learning?

RQ3: What training-time and communication overhead does the U-shaped architecture incur relative to conventional split learning, and is this overhead acceptable for real-world industrial deployment?

RQ4: Can a low-latency, low-cost blockchain ledger (e.g., Solana) be integrated with the predictive model to create an automated, closed-loop verification system that reliably tracks and validates the entire maintenance lifecycle—from initial fault detection to post-service repair verification?

III. METHODOLOGY

A. SYSTEM OVERVIEW

The proposed framework is designed for privacy-preserving distributed fault diagnosis in multi-organization industrial environments. As illustrated conceptually through the protocol descriptions below, the system comprises three geographically distributed client machines and a centralized coordination server. Each client node locally stores vibration sensor measurements from its assigned CNC milling machine and performs partial model training without transmitting raw data to any external entity. The server participates in collaborative training by processing intermediate feature representations forwarded by clients, but is architecturally prevented from observing either raw inputs or ground-truth labels.

To evaluate the effect of architectural design on predictive performance and privacy preservation, two split learning protocols are implemented and compared under identical experimental conditions:

- 1) **Normal Split Learning (NSL):** The conventional configuration in which clients transmit both activation tensors and labels to the server for loss computation.
- 2) **U-Shaped Split Learning (USL):** The proposed configuration in which labels never leave the client, and loss computation is performed locally using server-returned embeddings.

Both protocols employ identical datasets, preprocessing steps, random seeds, train-test partitions, model initialization procedures, optimization hyperparameters, and batch scheduling mechanisms, ensuring that any observed performance or privacy differences can be attributed solely to the architectural modification.

B. DATASET PREPARATION AND EXPERIMENTAL CONTROL

1) Bosch CNC Machining Dataset

The Bosch CNC Machining Dataset contains real-world vibrations from many industrial CNC milling machines and includes fifteen different kinds of machining operations (OP00–OP14) as well as vibration data collected in the X, Y and Z directions at 2 KHz by Bosch CISS (Connected Industrial Sensor Solution) tri-axial accelerometer sensors. The dataset includes both normal operating conditions and anomalous states caused by tool wear and machining irregularities, with binary labels designating each recording as normal or faulty. The dataset was selected for its industrial relevance, long term data collection spanning multiple machine operating periods and process variability across different machines and milling tasks making it both challenging and realistic for evaluating privacy preserving fault diagnosis systems.

2) Preprocessing and Client Partitioning

Raw vibration signals are divided into fixed-size windows of 1024 points, overlapping by 50%. Every window is independently normalized to have zero mean and unit variance along every axis in order to compensate for the amplitude shifts due to different sensor installation. These three-channel (X, Y, Z) segments are the input data used for the front model.

A global random seed remains constant during data loading, model initialization, and sample shuffling processes. Equivalent train/test partitions (80%/20%) are created using a common index creation technique. The data is segmented into three different disjoint sets related to machines M01, M02, and M03, which are allocated to clients 1, 2, and 3 correspondingly. Such segmentation is motivated by a real-world multi-organization setting where every client possesses data from its own set of machines without the intention of sharing the recordings.

Moreover, since industrial users might have different amounts of training data, an automated synchronous batching scheme is used in this system. In other words, the server dynamically sets the global batch size according to the minimum sample size from all the connected users. This helps to ensure that there is perfect synchronization among all organizations in the training loop without sharing information about their datasets except for just one number – the sample size.

C. MODEL ARCHITECTURE

A one-dimensional Convolutional Neural Network (1D-CNN) is employed for vibration-based fault analysis. 1D-CNNs automatically learn hierarchical temporal features from raw signals—including impulse patterns, spectral envelope modulations, and frequency components associated with machine faults—without requiring domain-expert feature engineering [4].

As illustrated in Figure 1, the network parameters are partitioned across the client and server. The overall network capacity is identical in both protocols; the architectural difference lies strictly in the physical location of the final

classification layers and the resulting forward/backward data flow.

1) Shared Front Model (Left Side: Client Model Part 1)

The front model is deployed on every client and performs local feature extraction from the raw input data (X). It corresponds to the “Left Side” of the architecture diagram and comprises two sequential convolutional blocks:

- **Block 1:** Conv1D(in_channels=3, out_channels=32, kernel_size=5), BatchNorm1D(32), ReLU, Max-Pool1D(kernel=2, stride=2)
- **Block 2:** Conv1D(in_channels=32, out_channels=64, kernel_size=3), BatchNorm1D(64), ReLU, Max-Pool1D(kernel=2, stride=2)

This model transforms each 3×1024 input window into an intermediate activation tensor of dimension 64×256 . These activations (denoted as $h_1, h_2, \dots$ in Figure 1) constitute the smashed data transmitted to the central server.

2) Middle Model (Center: Central Server Model)

The central server hosts the intermediate feature-refinement module. During the forward pass, it receives the parallel client activations (h), aggregates them in the batching block, and processes them through the following layers:

- Conv1D(in_channels=64, out_channels=128, kernel_size=3), BatchNorm1D(128), ReLU
- Global Average Pooling (reduces temporal dimension to 1)
- Fully Connected ($128 \rightarrow 64$), ReLU

This module refines the client-generated activations into a compact 64-dimensional embedding ($z_1, z_2, \dots$), capturing high-level fault features without the server ever accessing the raw input data (X).

3) Normal Split Learning Tail (Server-Side)

In the conventional NSL protocol, the classification head resides on the server immediately after the middle model:

- Fully Connected ($64 \rightarrow 1$)
- Sigmoid Activation

Because the output layer is on the server, clients must transmit their local labels (y) to the server to calculate the binary cross-entropy loss. The server then computes the loss and returns upstream gradients to the clients.

4) U-Shaped Split Learning Tail (Right Side: Client Model Part 2)

In the proposed USL protocol, the server does not hold the classification layers. Instead, it transmits the processed outputs (z) back to the client. This completes the forward pass and shifts the computation to the “Right Side” of the diagram, where a lightweight tail network performs the final classification:

- Dropout (rate = 0.3)
- Fully Connected ($64 \rightarrow 1$)

- Sigmoid Activation

The client calculates the binary cross-entropy loss locally using its private local labels (y). The client derives the gradients at the cut layer ($\nabla z_1, \nabla z_2, \dots$) and transmits only these gradients back to the server for the backward pass. This enforces the “U-Shaped Flow” constraint ensuring that labels never leave the client.

D. TRAINING PROTOCOLS

1) Data Flow in a Single Training Iteration

To provide an intuitive understanding of the U-Shaped Split Learning architecture prior to the formal algorithmic definitions, the flow of a single training iteration can be summarized in four sequential steps:

- 1) **Client Forward Pass:** Each participating client independently passes a local batch of raw vibration windows through its front model and transmits the resulting intermediate activation tensor (smashed data) to the server.
- 2) **Server Synchronization and Forward Pass:** The server acts as a synchronous coordinator. It **waits** to receive smashed data tensors from all clients, concatenates them into a single global batch, and processes this combined batch through the shared middle model. The resulting embeddings are then split, and each client receives its respective portion.
- 3) **Client Loss and Backward Pass:** Each client passes its received embedding through its local tail network to yield the final prediction. It then computes the binary cross-entropy loss against its local ground-truth labels. The client performs backpropagation through the tail network, calculates the gradient of the loss with respect to the embedding, and transmits this embedding gradient back to the server.
- 4) **Server Synchronization and Backward Pass:** The server again **waits** to receive embedding gradients from all clients, concatenates them, and uses this stacked gradient to update the middle model’s weights. It then computes the upstream gradient with respect to the smashed data, splits it, and returns the respective portions to the clients to complete backpropagation for their front models.

2) Normal Split Learning Protocol

Algorithm 1 formalizes the conventional split learning training loop, and the corresponding network topology is illustrated in Fig. 2.

In NSL, label y travels to the server in every iteration, and the returned gradient $g_A = \partial \mathcal{L} / \partial A$ carries label-dependent information. This exposes the client to both label disclosure and gradient inversion risks.

3) Proposed U-Shaped Split Learning Protocol

Algorithm 2 formalizes the proposed U-shaped protocol, with its architectural flow depicted in Fig. 3.

Algorithm 1 Normal Split Learning (NSL) Training

Require: Clients $\{C_i\}_{i=1}^N$, Server S , dataset $\{(X_i, Y_i)\}$, epochs T , learning rate η
Ensure: Trained front models $\{\theta_i^{\text{front}}\}$, server model θ^{server}

```

1: for epoch  $t = 1$  to  $T$  do
2:   for each mini-batch  $(x, y)$  across clients do
3:     // — Client Forward Pass —
4:      $A \leftarrow f_{\text{front}}(x; \theta_i^{\text{front}})$   $\triangleright$  Compute smashed data
5:     Send  $(A, y)$  to server  $S$ 
6:     // — Server Forward & Loss —
7:      $\hat{y} \leftarrow f_{\text{server}}(A; \theta^{\text{server}})$   $\triangleright$  Full forward pass
8:      $\mathcal{L} \leftarrow \text{BCE}(\hat{y}, y)$   $\triangleright$  Loss with transmitted labels
9:     // — Server Backward Pass —
10:    Compute  $\nabla_{\theta^{\text{server}}} \mathcal{L}$  and update  $\theta^{\text{server}}$ 
11:     $g_A \leftarrow \partial \mathcal{L} / \partial A$   $\triangleright$  Upstream gradient for client
12:    Send  $g_A$  to client  $C_i$ 
13:    // — Client Backward Pass —
14:    Update  $\theta_i^{\text{front}}$  using  $g_A$  via chain rule
15:   end for
16: end for
```

Algorithm 2 U-Shaped Split Learning (USL) Training

Require: Clients $\{C_i\}_{i=1}^N$, Server S , dataset $\{(X_i, Y_i)\}$, epochs T , learning rate η
Ensure: Trained front models $\{\theta_i^{\text{front}}\}$, server middle model θ^{mid} , client tail models $\{\theta_i^{\text{tail}}\}$

```

1: for epoch  $t = 1$  to  $T$  do
2:   for each mini-batch  $(x, y)$  across clients (label  $y$  stays local) do
3:     // — Client Forward Pass (Front) —
4:      $A \leftarrow f_{\text{front}}(x; \theta_i^{\text{front}})$   $\triangleright$  Smashed data only, no label
5:     Send  $A$  to server  $S$   $\triangleright y$  is never sent
6:     // — Server Forward Pass (Middle) —
7:      $E \leftarrow f_{\text{mid}}(A; \theta^{\text{mid}})$   $\triangleright$  64-dim embedding
8:     Send  $E$  back to client  $C_i$ 
9:     // — Client Forward Pass (Tail) + Loss —
10:     $\hat{y} \leftarrow f_{\text{tail}}(E; \theta_i^{\text{tail}})$   $\triangleright$  Local classification
11:     $\mathcal{L} \leftarrow \text{BCE}(\hat{y}, y)$   $\triangleright$  Local loss—label never transmitted
12:    // — Client Backward Pass (Tail) —
13:    Compute  $\nabla_{\theta_i^{\text{tail}}} \mathcal{L}$  and update  $\theta_i^{\text{tail}}$ 
14:     $g_E \leftarrow \partial \mathcal{L} / \partial E$   $\triangleright$  Embedding gradient to server
15:    Send  $g_E$  to server  $S$ 
16:    // — Server Backward Pass (Middle) —
17:    Compute  $\nabla_{\theta^{\text{mid}}} \mathcal{L}$  using  $g_E$  and update  $\theta^{\text{mid}}$ 
18:     $g_A \leftarrow \partial \mathcal{L} / \partial A$  via  $g_E$  and  $\theta^{\text{mid}}$   $\triangleright$  Upstream gradient
19:    Send  $g_A$  to client  $C_i$ 
20:    // — Client Backward Pass (Front) —
21:    Update  $\theta_i^{\text{front}}$  using  $g_A$  via chain rule
22:   end for
23: end for
```

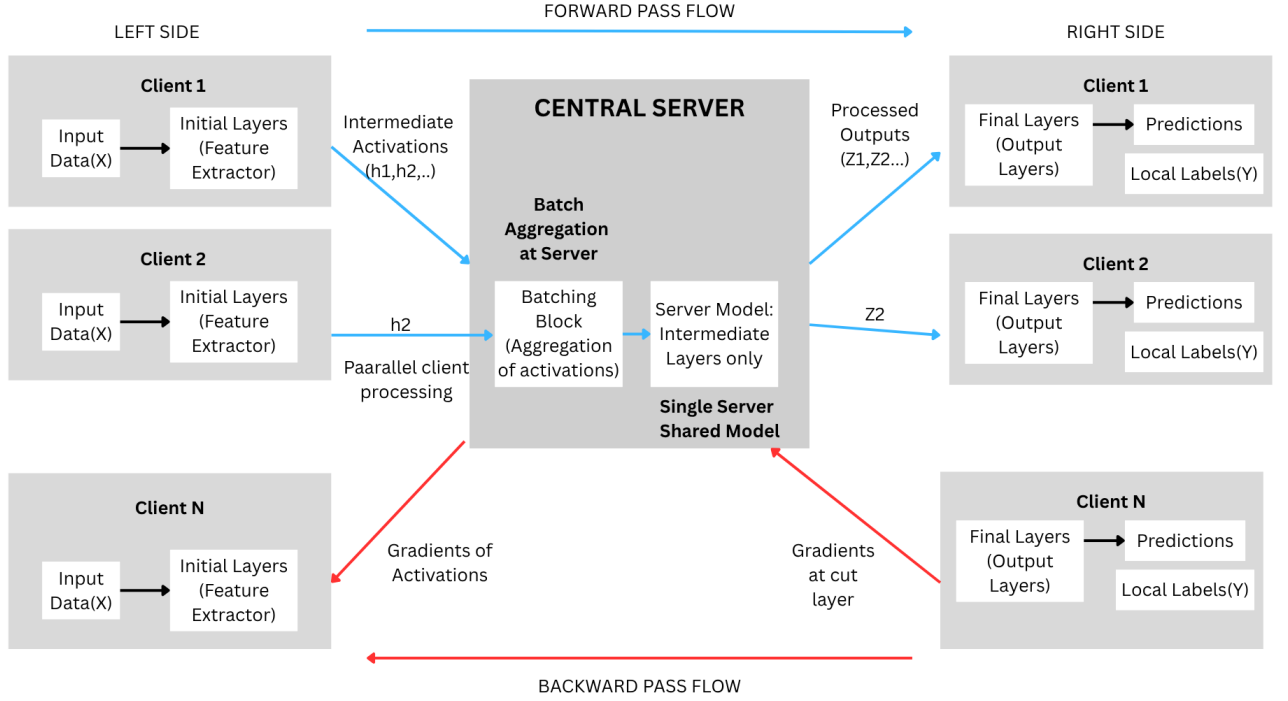


FIGURE 1. U-Shaped Parallel Split Learning Architecture mapping the forward pass of raw data (X) to intermediate activations (h) and processed outputs (z), followed by the backward pass of cut-layer gradients and activation gradients.

The key distinction is that y is used only in Step 10 and never transmitted. The server computes g_A using g_E and the middle model parameters, but g_E does not carry label information because $\mathcal{L}$ was computed on the client after the embedding was returned. Consequently, both the Information Exposure Ratio and the Gradient Inversion Risk Score are structurally zero in USL.

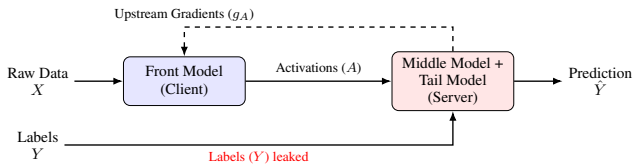


FIGURE 2. Architecture of Normal Split Learning (NSL). The raw data X stays on the client, but the ground-truth labels Y must be transmitted to the server for loss computation, creating a privacy vulnerability.

E. PRIVACY EVALUATION METRICS

Three complementary quantitative metrics are used to evaluate privacy leakage at the client-server communication boundary.

1) Information Exposure Ratio (IER)

The IER measures the fraction of label entropy $H(Y)$ exposed to the server through transmitted messages in each training iteration:

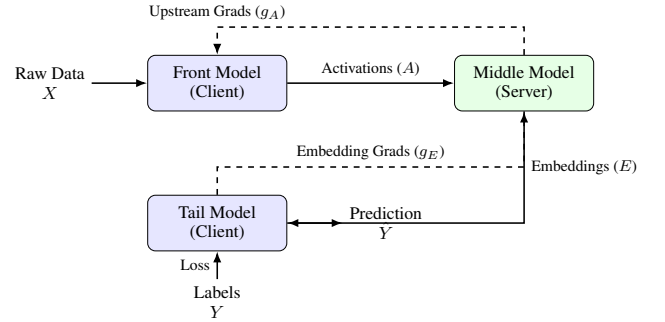


FIGURE 3. Architecture of U-Shaped Split Learning (USL). The forward pass forms a "U" shape, returning embeddings to the client. Labels Y are kept strictly private on the client for local loss computation.

$$\text{IER} = \frac{H(Y \mid \text{server-observable information})}{H(Y)} \quad (1)$$

In NSL, labels are transmitted with every batch, so the server observes Y directly, yielding $\text{IER} = 1.0$. In USL, no label-related information is transmitted to the server, so $\text{IER} = 0.0$ by construction.

2) Reconstruction Normalized Mean Squared Error (NMSE)
To quantify input reconstruction risk, an adversary decoder network is trained for 30 epochs to reconstruct the raw client input X from the activation tensor A transmitted to the server:

$$\text{NMSE} = \frac{\text{MSE}(X, \hat{X})}{\text{Var}(X)} \quad (2)$$

where X and $\hat{X}$ denote the original and reconstructed vibration window, respectively. A higher NMSE indicates that the activation tensor is more difficult to invert, corresponding to stronger protection against input reconstruction attacks. The decoder architecture is a symmetric transposed-convolutional network with the same layer count as the front model.

3) Gradient Inversion Risk Score (GIRS)

GIRS quantifies the magnitude of label-dependent gradients returned to the client:

$$\text{GIRS} = \mathbb{E} \left[\left\| \frac{\partial \mathcal{L}}{\partial A} \right\|_2 \right] \quad (3)$$

where $\mathcal{L}$ is the training loss and A is the activation tensor at the split layer. Non-zero GIRS values indicate the presence of gradients that embed supervisory label information and could potentially be exploited by gradient inversion attacks [6], [8]. In USL, the server never computes $\mathcal{L}$ and therefore never produces $\partial \mathcal{L} / \partial A$; the gradient g_A returned to the client is derived purely from the middle model parameters and g_E . While g_E is mathematically influenced by the label, it does not carry a direct label signal, meaning the server observes no direct supervisory information. Accordingly, USL structurally eliminates direct label exposure, yielding a theoretical GIRS of 0.0 with respect to direct loss gradients.

F. BLOCKCHAIN-BASED MAINTENANCE LOGGING

1) Motivation and Design Rationale

While USL protects data privacy in the training process, the maintenance process raises another issue that needs to be resolved: the integrity of fault logs, maintenance logs, and verification results needs to be guaranteed for accountability throughout the industrial supply chain. Traditional relational databases are vulnerable to the risks of record alteration, record deletion, and single point of failure. The use of blockchain technology eliminates these issues through the use of append-only, cryptographically connected record. BI [26], [29].

2) Solana Smart Contract Architecture

The blockchain component is deployed on the Solana Devnet using the Anchor development framework. Solana's Proof-of-History (PoH) consensus mechanism achieves high transaction throughput (theoretical maximum 65,000 TPS) with sub-second finality, making it suitable for real-time industrial maintenance logging where service records must be generated with minimal latency [31]. The Borsh (Binary Object Representation Serializer for Hashing) serialization protocol encodes maintenance records compactly before submission to the network, minimizing per-transaction byte footprint.

Each maintenance record stores the following fields: machine identifier, log type (FAULT, SERVICE, VERIFICATION), descriptive message, Unix timestamp, and transaction metadata (slot number, block hash). Records are never updated in place; instead, each subsequent action—service attempt, technician sign-off, verification outcome—creates a new linked transaction that references the original fault record's on-chain address.

3) End-to-End Fault Lifecycle

Algorithm 3 describes the complete blockchain-integrated anomaly response workflow.

Algorithm 3 Blockchain-Integrated Anomaly Response Workflow

Require: Anomaly score s from USL model, threshold τ , Solana program $\mathcal{P}$

- 1: **if** $s > \tau$ **then** ▷ Anomaly detected
- 2: Create fault record: $r_0 \leftarrow \{\text{machine_id}, \text{"FAULT"}, \text{description}, \text{timestamp}\}$
- 3: Submit r_0 to $\mathcal{P}$ via Node.js service ▷ Transaction TX_0
- 4: Retrieve on-chain address $\alpha_0 \leftarrow \text{TX}_0.\text{address}$
- 5: Dispatch maintenance alert to assigned technician
- 6: **// Service event (upon technician action)**
- 7: Create service record: $r_1 \leftarrow \{\text{machine_id}, \text{"SERVICE"}, \text{action}, \text{timestamp}, \text{ref: } \alpha_0\}$
- 8: Submit r_1 to $\mathcal{P}$ ▷ Transaction TX_1
- 9: **// Verification event**
- 10: **if** service verified successful **then**
- 11: Create record: $r_2 \leftarrow \{\text{machine_id}, \text{"VERIFIED"}, \text{outcome}, \text{timestamp}, \text{ref: } \alpha_0\}$
- 12: **else**
- 13: Create record: $r_2 \leftarrow \{\text{machine_id}, \text{"PENDING_VERIFICATION"}, \text{notes}, \text{timestamp}, \text{ref: } \alpha_0\}$
- 14: **end if**
- 15: Submit r_2 to $\mathcal{P}$ ▷ Transaction TX_2
- 16: **end if**

The complete audit trail for a fault lifecycle thus consists of an immutable sequence $\text{TX}_0 \rightarrow \text{TX}_1 \rightarrow \text{TX}_2$ on the Solana ledger, providing cryptographically verifiable provenance for every maintenance action. Because Solana records are append-only and consensus-protected, even a compromised Node.js service cannot retroactively modify previously submitted records. Crucially, the “Verification event” is an automated, closed-loop process: once a technician logs a repair, the system forces the CNC machine to run for N predefined cycles while being monitored by the anomaly detection model. Only if the model confirms the absence of anomalous vibration patterns is the service marked as successful on the blockchain, eliminating human error or premature sign-offs from the maintenance lifecycle.

IV. RESULTS AND DISCUSSION

A. EXPERIMENT SETUP

The experiments were all performed in a simulated multi-client environment running on commodity hardware. Three client processes represent machines M01, M02, and M03 respectively, each communicating with a single server process over a local network with artificially imposed latency to simulate WAN conditions. The NSL and USL models were trained for 30 epochs (having a batch size of 64), applying the Adam optimizer with learning rate 10^{-3} , $\beta_1 = 0.9$, $\beta_2 = 0.999$, and weight decay 10^{-4} . For loss function, binary cross-entropy was used in both protocols. The decoder for NMSE evaluation is a five-layer transposed-convolutional network trained with mean squared error loss for 30 epochs.

B. CLASSIFICATION PERFORMANCE

Table 1 summarizes mean classification performance across all three clients for both protocols.

TABLE 1. Classification Performance Comparison (Mean across 3 Clients)

Method	Accuracy (%)	F1-Score (%)	Time (s)	Comm. (MB)
Normal Split (NSL)	86.46	78.57	250.81	7056.40
U-Shape Split (USL)	87.44	80.82	263.53	7083.87

The proposed USL framework achieves an average accuracy of 87.44% and F1-score of 80.82%, slightly surpassing NSL (86.46% accuracy, 78.57% F1-score), as visually compared in Fig. 4. This performance improvement, while modest in absolute terms, is noteworthy because it demonstrates that privacy enhancement and predictive capability are not in conflict—the U-shaped topology can simultaneously strengthen privacy guarantees and modestly improve classification performance.

The increase in training time from 250.81 s to 263.53 s (approximately 5.1%, shown in Fig. 6) and communication overhead from 7056.40 MB to 7083.87 MB (approximately 0.4%, shown in Fig. 5) are both negligible in the context of real-world industrial deployments, where training sessions are expected to run over extended periods. The small communication overhead increase in USL is attributable to the additional round trip in which the server returns the embedding E to the client before loss computation, as well as the transmission of g_E in the backward pass.

C. PRIVACY ANALYSIS

Table 2 presents the three privacy metrics for both protocols.

TABLE 2. Privacy Metric Comparison

Method	IER	Recon. NMSE	GIRS
Normal Split (NSL)	1.0	0.0392	0.0087 ± 0.0142
U-Shape Split (USL)	0.0	0.0429	0.0000

1) Information Exposure Ratio

As illustrated in Fig. 7, the IER result is the most fundamental distinction between the two protocols. In NSL, labels are transmitted to the server with every training batch,

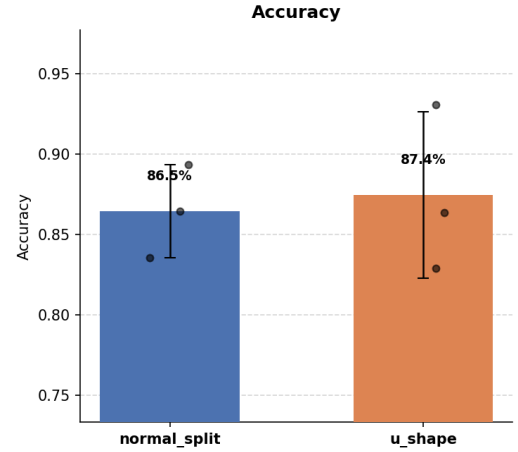


FIGURE 4. Per-client accuracy comparison between Normal Split Learning (NSL) and U-Shaped Split Learning (USL) across clients M01, M02, and M03.

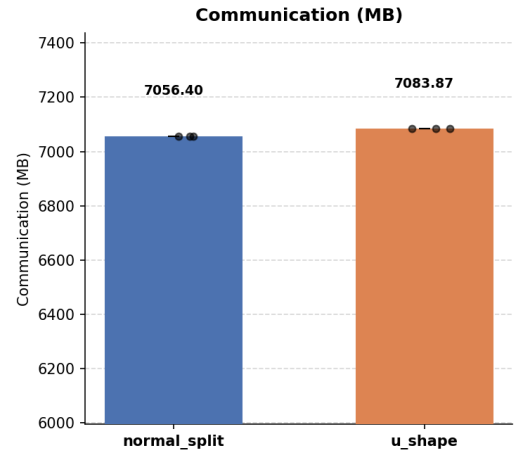


FIGURE 5. Communication overhead comparison between NSL and USL (total data exchanged over the training period).

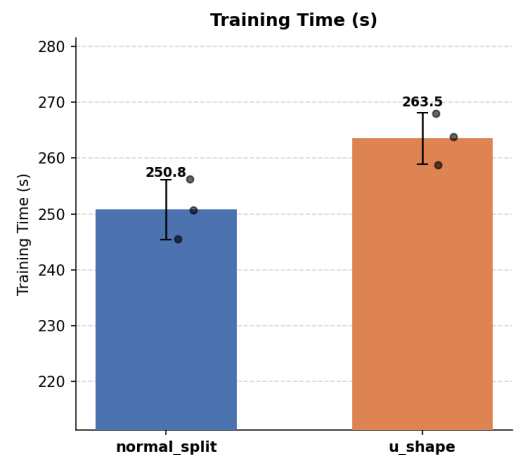


FIGURE 6. Total training time comparison between NSL and USL.

yielding $IER = 1.0$ —complete supervisory information exposure. In USL, labels are used exclusively in the client-side loss computation and are never transmitted, yielding $IER = 0.0$ by architectural construction. This eliminates the most direct privacy risk in conventional SL: an adversary who compromises the server in NSL has immediate access to all client labels, enabling trivial label inference attacks. In USL, such an adversary gains no label information from server-observable traffic.

2) Reconstruction NMSE

As shown in Fig. 8, the adversary decoder achieves a higher NMSE on USL activations (0.0429) than on NSL activations (0.0392). Although both values indicate that a moderately trained decoder can partially reconstruct vibration signals from smashed data, the higher NMSE in USL suggests that the front model trained under USL generates activations that are somewhat less invertible. This is consistent with the expectation that the more distributed gradient flow in USL pushes the front model to learn representations which are more information-efficient with respect to the input, rather than maximally preserving input-level features as in NSL.

3) Gradient Inversion Risk Score

GIRS is reduced to exactly zero in USL. Because the server never computes the loss function, the gradient g_A returned to the client is derived solely from the middle model's chain-rule computation using g_E . The embedding gradient g_E , generated on the client, does not carry direct label information since the tail network's sigmoid activation and binary cross-entropy loss involve both the prediction $\hat{y}$ and the label y only locally. In contrast, NSL's $g_A = \partial \mathcal{L} / \partial A$ is computed by the server with full knowledge of y , yielding a non-zero GIRS of 0.0087 ± 0.0142 across training batches. The non-zero standard deviation reflects variation in gradient magnitudes across batches with different class balances.

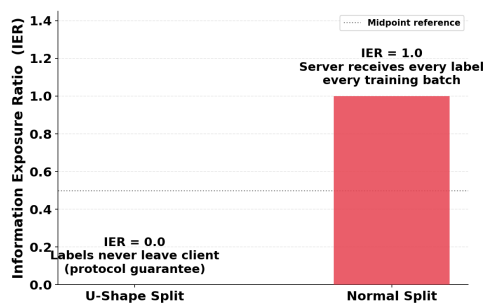


FIGURE 7. Information Exposure Ratio (IER) comparison. NSL exposes complete label information ($IER = 1.0$); USL achieves zero label leakage ($IER = 0.0$) by design.

D. BLOCKCHAIN PERFORMANCE EVALUATION

The blockchain logging module was deployed on the Solana Devnet using the Anchor smart contract framework. Table 3 summarizes the observed performance metrics.

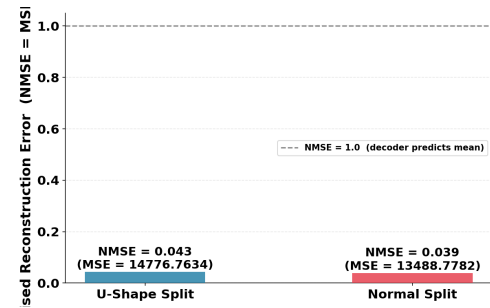


FIGURE 8. Reconstruction NMSE comparison. Higher NMSE indicates greater resistance to adversarial input reconstruction from transmitted activations.

TABLE 3. Blockchain Performance Evaluation

Metric	Observed Value
Blockchain Platform	Solana Devnet
Smart Contract Framework	Anchor
Serialization Protocol	Borsh
Average Confirmation Latency	400–600 ms
Transaction Cost	<0.00001 SOL
Record Update Strategy	Append-only
Fault Lifecycle States	FAULT → SERVICE → VERIFIED / PENDING

Average transaction confirmation latency of 400–600 ms is sufficiently low for industrial maintenance workflows, where a few hundred milliseconds of logging delay is inconsequential relative to the minutes or hours required for physical servicing. The per-transaction cost below 0.00001 SOL (approximately \$0.001–\$0.01 at typical SOL valuations) makes the system economically viable even for high-frequency fault detection deployments. Each fault lifecycle is fully represented by an immutable chain of transactions, providing a verifiable audit trail that can be queried by maintenance engineers, compliance auditors, and insurance investigators without relying on a centrally controlled database.

E. DISCUSSION AND LIMITATIONS

The experimental results confirm that U shape split learning effectively eliminates the two primary privacy leakage channels of conventional SL which include label transmission and label-dependent gradient exposure. At the same time it achieves comparative classification performance as Normal Split learning. The reconstruction NMSE improvement is more modest which reflecting the fact that USL's front model still transmits activation tensors that partially encode input information. This residual leakage is an inherent property of split learning and cannot be eliminated without additional mechanisms such as differential privacy noise injection or activation compression.

Several limitations of the current study are to be acknowledged. Firstly, split learning environment was simulated by representing multiple clients on a single computing platform rather than deploying physically distributed client devices. This in turn may not fully capture real-world network latency and jitter effects. Second, the privacy analysis assumes an honest-but-curious server adversary; a fully malicious server that deviates from the agreed protocol (e.g. by training a

reconstruction model on intercepted activations) may extract more information than our metrics capture. Third, blockchain verification in the prototype relies on controlled simulation rather than real CNC machines since physical equipment was unavailable.

Future work will address these limitations by extending this framework to larger multi-client deployments. Physical network infrastructure will also help us integrate differential privacy noise into the activation tensors to limit the risk of residual data reconstruction and testing the framework against an adversarial server model. Additionally, the blockchain integration will be expanded to include the Solana Mainnet and will be evaluated on the basis of real world transaction costs and overall network congestion.

V. CONCLUSION

In this paper we present a framework for Privacy-Preserving U-Shaped Split Learning (USL) to diagnose faults in CNC machines and validate it using actual tri-axial vibration data collected from real-world CNC machining systems. One of the key issues with standard Split Learning is that the server needs to have access to the labels, and we solve this problem by using a U-shaped architecture: The first several and last several layers of the neural network will reside in the client, while all middle layers will reside on the server. This solution allows for the entire raw vibration dataset and actual labels to be retained at the client, which prevents any revealing of the dataset or labels to the server.

We tested this setup on the Bosch CNC Machining Dataset, and USL gave us a mean classification accuracy of 87.44% with an F1-score of 80.82%, which is a small improvement over conventional Split Learning (86.46%, 78.57%). More importantly, USL completely removes label exposure (IER goes from 1.0 to 0.0), brings gradient inversion risk down to zero (GIRS from 0.0087 to 0.0000), and is also somewhat more resistant to input reconstruction attacks (NMSE 0.0392 to 0.0429). All these privacy gains only cost us a 5.1% increase in training time and just 0.4% more communication overhead, so the approach is practical to actually use.

In addition, we added a second layer to the system by creating a blockchain solution on Solana to make an end to end system around all maintenance of the machine to form an immutable and cryptographically secured record of everything that occurs in that system throughout the entire lifecycle of the machine. In addition to logging all activities, we will also utilize our anomaly detection system to identify whether or not the technician's repairs corrected the issue, and generate a final validation transaction only after the machine has returned to its normal vibration pattern. Transactions on the blockchain will typically be verified in less than 1 second (401 - 599 msec), with almost no transaction costs, which enables excellent auditability while still not limiting industrial processes to real-time operations.

The framework is useful for cases where there are multiple organisations, manufacturers or suppliers involved in training a machine learning model together without sharing their

datasets with each other. Next steps include scaling to larger multi-client IIoT environments, adding Differential Privacy to mitigate the possibility of leakage from the activation level, evaluating how well the system performs with fully malicious servers and transitioning the Blockchain component onto the Solana mainnet to see if it is ready for production use.

REFERENCES

- [1] E. Johnson, "The true cost of downtime," Siemens Blog, Apr. 2023. [Online]. Available: <https://blog.siemens.com/2023/04/the-true-cost-of-downtime/>
- [2] Y. Lei et al., "Applications of machine learning to machine fault diagnosis: A review and roadmap," *Mech. Syst. Signal Process.*, vol. 138, p. 106587, 2020.
- [3] G. Niu et al., "Enhanced discriminate feature learning deep residual CNN for multi-task bearing fault diagnosis with information fusion," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 762–770, 2023.
- [4] W. Zhang et al., "A new deep learning model for fault diagnosis with good anti-noise and domain adaptation ability on raw vibration signals," *Sensors*, vol. 17, no. 2, p. 425, 2017.
- [5] B. McMahan et al., "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2017, pp. 1273–1282.
- [6] J. Zhao et al., "iDLG: Improved deep leakage from gradients," *arXiv preprint arXiv:2001.02610*, 2020.
- [7] P. Vepakomma et al., "Split learning for health: Distributed deep learning without sharing raw patient data," *arXiv preprint arXiv:1812.00564*, 2018.
- [8] L. Zhu et al., "Deep leakage from gradients," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 14747–14756.
- [9] H. Yin et al., "See through gradients: Image batch recovery via GradInversion," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 16332–16341.
- [10] M. Zhao et al., "RUL prediction of turbofan engine based on LSTM-transformer model," *Sensors*, vol. 22, no. 16, p. 6225, 2022.
- [11] S. Otoum et al., "On the feasibility of split learning, transfer learning and federated learning for preserving security in ITS systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 7, pp. 7462–7470, 2022.
- [12] R. Bharti and A. McGibney, "Privacy-aware resource sharing in cross-device federated model training for collaborative predictive maintenance," *IEEE Internet Things J.*, 2021.
- [13] Z. Hu et al., "A review and experimental evaluation on split learning," *Future Internet*, vol. 17, no. 2, p. 87, 2025.
- [14] W. Wu et al., "Split learning over wireless networks: Parallel design and resource management," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 4, pp. 1051–1066, 2023.
- [15] W. Ni et al., "Federated split learning for resource-constrained robots in industrial IoT: Framework comparison, optimization strategies, and future directions," *arXiv preprint arXiv:2501.xxxxx*, 2025.
- [16] C. Thapa et al., "SplitFed: When federated learning meets split learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 8, 2022, pp. 8485–8493.
- [17] H. Hafi et al., "Split federated learning for 6G enabled-networks: Requirements, challenges, and future directions," *IEEE Access*, vol. 12, pp. 9890–9930, 2024.
- [18] S. Sindhura, B. Annappa, and D. N. Sachin, "Federated split learning with HyperNetworks for medical image classification," in *Proc. 15th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, 2024.
- [19] H. Tang et al., "Edge-assisted U-shaped split federated learning with privacy-preserving for Internet of Things," *Expert Syst. Appl.*, vol. 255, p. 124555, 2024.
- [20] Z. Yang et al., "Dynamic corrected split federated learning with homomorphic encryption for U-shaped medical image networks," *IEEE J. Biomed. Health Informat.*, 2023.
- [21] J. Liu et al., "Similarity-based label inference attack against training and inference of split learning," *IEEE Trans. Inf. Forensics Security*, 2024.
- [22] J. Lin et al., "Efficient and privacy-preserving inference for medical images via FHE-enhanced split neural networks," in *Proc. IEEE Int. Conf. Bioinformatics and Biomedicine (BIBM)*, 2025.
- [23] A. Alhindi et al., "Advancements and challenges in privacy-preserving split learning: Experimental findings and future directions," *Int. J. Inf. Secur.*, 2025.

- [24] F. Khan et al., "Federated split learning model for industry 5.0: A data poisoning defense for edge computing," *Electronics*, vol. 11, no. 15, p. 2393, 2022.
- [25] A. Yazdinejad et al., "Block Hunter: Federated learning for cyber threat hunting in blockchain-based IIoT networks," *IEEE Trans. Ind. Informat.*, vol. 18, no. 11, pp. 8356–8366, 2022.
- [26] J. Weng et al., "DeepChain: Auditable and privacy-preserving deep learning with blockchain-based incentive," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2438–2455, 2021.
- [27] Y. Miao et al., "Privacy-preserving Byzantine-robust federated learning via blockchain systems," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2848–2861, 2022.
- [28] J. Chen et al., "Privacy-preserving and traceable federated learning for data sharing in industrial IoT applications," *Expert Syst. Appl.*, vol. 213, p. 119036, 2023.
- [29] A. Ahmad et al., "BlockTrail: A service for secure and transparent blockchain-driven audit trails," *IEEE Syst. J.*, vol. 16, no. 1, pp. 1367–1378, 2022.
- [30] X. Jin et al., "Distributed IIoT anomaly detection scheme based on blockchain and federated learning," *J. Commun. Netw.*, vol. 26, no. 2, pp. 252–262, 2024.
- [31] Y. Yakovenko, "Solana: A new architecture for a high performance blockchain," *Solana Technical Whitepaper*, 2020. [Online]. Available: <https://solana.com/solana-whitepaper.pdf>



SURYA PRAKASH GARLAPATI is currently pursuing the B.E. degree in computer science and engineering at RV College of Engineering, Bengaluru, India. His major field of study is computer science and engineering, and he is currently in his second year of undergraduate study.

His research interests include federated learning, deep learning, and predictive maintenance. He is eager to gain hands-on experience, collaborate with innovative teams, and contribute to impactful

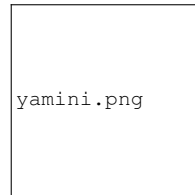
projects while building a successful career in technology.

He can be contacted via email at suryaprakashg.cs23@rvce.edu.in.



VINAYAK KUMAR SINGH is an undergraduate student in Information Science and Engineering at RV College of Engineering, Bengaluru. His academic interests include software engineering, applied cryptography, and Internet of Things (IoT) systems. He is currently focused on backend development and algorithmic optimization.

He can be contacted via email at ogv-inayak@gmail.com.



N YAMINI is currently pursuing the Bachelor of Engineering degree in Artificial Intelligence and Machine Learning at RV College of Engineering, Bengaluru, India. Her research interests include artificial intelligence, machine learning, cybersecurity, and computer vision. She is passionate about developing secure, scalable, and practical AI solutions for real-world applications.



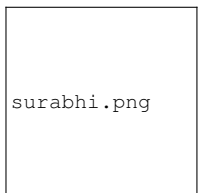
TANISHA SRIVASTAVA is currently pursuing the Bachelor of Engineering degree in Computer Science and Engineering at RV College of Engineering, Bengaluru, India. Her research interests include deep learning, natural language processing and algorithm design. She is passionate about developing secure, scalable, and practical computing solutions that address real-world challenges through innovative technologies.

She can be contacted via email at tanishasri21@gmail.com



DR. ANALA M. R. is working as a Professor at R.V. College of Engineering and is an IEEE member. She has over 20+ years of experience in teaching and research. Her area of research includes Deep Learning, Computer Architecture, High Performance Computing, Distributed Systems, and Parallel Programming. She has guided more than 70 UG and PG projects. She has published over 70+ papers in national and international journals and conferences.

...



SURABHI R is currently pursuing the Bachelor of Engineering degree in Computer Science and Engineering at RV College of Engineering, Bengaluru, India. Her research interests include blockchain, data analytics and distributed systems. She is passionate about developing secure, scalable, and innovative technologies that address real-world challenges across diverse application domains.

She can be contacted via email at surabhi.r052@gmail.com